

Lab 07

Seti Lab: Quickly Searching for Extraterrestrial Life

Points: 100

Credits: Branden Ghena (Northwestern), Surya Gunukula

Table of Contents

<i>Goals of the Lab Assignment</i>	1
<i>High Level Summary of Entire Lab</i>	2
<i>Introduction</i>	3
<i>STEP 0: Getting Started</i>	4
<i>STEP 1: Understanding Sequential Band Scan</i>	5
<i>STEP 2: P_BAND_SCAN.C</i>	6
<i>STEP 3: Submitting Via Slurm</i>	7
<i>APPENDIX:</i>	8
<i>A. Background Knowledge</i>	
<i>a. Signal Processing</i>	8
<i>b. Regular Unix I/O and Memory-Mapped I/O</i>	12
<i>c. Pthreads and Processor Affinity</i>	12
<i>d. Measuring Time, Performance, and Resource Usage</i>	13
<i>B. Advice</i>	13

Goals of the Lab Assignment

The goals of this lab are to:

1. Get you to think about what goes into a simple parallel algorithm
2. Expose you to low-level parallel programming (C) using pthreads
3. Expose you to Unix I/O programming interface
4. Understanding and improving on large codebases

High-Level Summary of the Lab

This lab is slightly different and more complicated than other labs you have done in this class, so here is a high-level overview of everything that happens in this lab, so that when you read the rest of this writeup, you don't get lost.

Step 1 of this lab explains the entire codebase. It walks you through the workflow of signal processing which is given to you, and by looking at all these programs you will learn a general understanding of the entire lab. But, pay special attention to **band_scan.c** because the entire lab is focused on creating a **parallelized version** of this. The rest of the files should remain unchanged and therefore don't need as detailed attention.

Then, with your understanding of parallelization and `band_scan.c`, you must create a file called **p_band_scan.c** that does EXACTLY what `band_scan.c` does but with parallelization. There are many avenues and opportunities for optimization in this code, so there will be a leaderboard for the fastest time. But to get full credit on this lab, you just need to achieve a similar or better time to a very naive solution.

The appendix is optional to read, but is HIGHLY recommended as it gives key information that could be useful when tackling this lab. Read it before attempting the lab and also look through whenever you're stuck or confused during any part of the lab.

Introduction

The Search for Extraterrestrial Intelligence (SETI) is a long-standing international effort to try to find alien civilizations (yes, little green men) through their radio wave (and more recently, optical) emissions. An inhabited planet like ours is as bright as a small star in the radio spectrum due to humanity's numerous radio transmitters, from big AM, FM, shortwave, and TV stations all the way down to the tiny transmitters used by computers and phones to connect to WiFi and cellular access points. We make full use of this spectrum to communicate with one another. One of the basic (and fairly naïve) assumptions of early SETI efforts was that this would hold true for other intelligent civilizations. They may, in fact, communicate using technology built on wild physics that we don't yet understand. But for now, we'll have to see what we can uncover with radio waves.

SETI uses antennas generally employed for radio astronomy to “look” at other stars, recording signals over an interval of time. Unlike a regular radio receiver, these receivers do not tune to a specific frequency, but rather listen on many frequencies at once—they are broadband receivers. They also do not try to decode or demodulate the signals they receive at all. The point is to produce huge amounts of signal data. The signal data contains not only possible alien signals, but also many human signals, and a lot of noise from all kinds of natural radio sources (e.g. the supermassive black hole at the center of our galaxy).

The signal data from the radio telescopes is analyzed with computer programs that use signal processing algorithms, which sift through the data looking for artificial signals that do not come from our own planet. This is a huge computational (and conceptual) challenge, and a significant breakthrough came with the SETI@HOME project, which allowed ordinary people to volunteer their computers to participate in a large-scale, distributed version of these programs. At its peak, there were over 1.3 million registered users whose computers contributed an average of about two thousand trillion floating point operations per second to the effort (2 PetaFLOPS!).

In this lab, you will tackle a greatly simplified and constrained version of this computation, with the goal of trying to make your computer execute the computation as fast as it can. You will write a program that, given a raw signal from a broadband receiver, will try to hunt for signs of intelligence within it, specifically some kind of message. Your program will determine whether a given frequency band has unusually high power, and if so, which band it is in. In addition, your program will use another program that we provide to determine whether that high-power signal actually appears to come from an intelligent source. For fun, we give you some other programs too; an AM demodulator and an audio converter, which given the signal and the band, will convert the signal to audio format and let you hear it. The AM demodulator is basically a simple AM radio, and your program will tell us how to tune it.

STEP 0: Getting Started

This lab must be done on a specific VM, not CSIL.

- 1) SSH into this VM using **ssh uname@cmpsc32.cs.ucsb.edu**. (This VM has the same shared filesystem as CSIL, but is needed for grading using SLURM)
- 2) Run this command:

```
cp -r /cs/faculty/nabeeln/public/cs32/labs/setilab-handout ./
```

We strongly recommend that you turn this into a git repository before you begin. Debugging parallel code is notoriously difficult, and being able to quickly rollback to a known-good version of your code makes debugging easier.

Your unzipped files will include:

- **README** — describes any last minute details
- **Makefile**
- **Scripts**
 - Bash scripts to submit your final code for performance and accuracy evaluation
- **Generation and Playback Programs**
 - **siggen**: this binary program will generate a signal for you to look at.
 - **amdemod**: this binary program will demodulate the signal and create an audio signal in a binary format
 - **bin2wav**: this binary program will translate the audio signal to a WAV file, which can be played back with any media player.
 - **analyze_signal**: this program looks for “information content” of your signal, i.e. language.
- **Signal and Filtering Code**
 - **signal.[ch]** load and store signal files in several ways, including text I/O using the stdio system, binary I/O using the Unix open/seek/read/write/close system call interface, and binary I/O using the Unix mmap system call interface.
 - **filter.[ch]** generate filters and apply them to signals (sequential only)
 - **timing.[ch]** find out how long things take using various methods
- **band_scan.c — a sequential version of the program you’ll write**
- **Pthread examples**
 - **pthread-ex.c** how to create and use threads on Unix (and most platforms)]
 - **parallel-sum-ex.c** how to compute the sum of an array in parallel

STEP 1: Understanding Sequential Band Scan

1. Generate your own signal files

Your first task is to get some signal data from our server. If this were actually SETI@Home, this would be real radio telescope data made available via a project server. For this lab, we've set up a system that generates virtual signals, akin to what you might see if you pointed a radio telescope out into space. Whereas SETI@Home looks for signals across a 2.5 MHz wide frequency band, we are giving you signals across just a 200 kHz frequency band, to make things more tractable. The signals comprise roughly 30 seconds of data. Typically, these radio signals would be subject to the Earth's rotation. However, to simplify things for you, we will ignore this effect. You can assume that your signal was recorded while the radio telescope was stationary and pointing at a fixed patch of sky.

To get your signal, run the `siggen` binary we provide:

```
setilab-handout> ./siggen
```

Note: The gradescope test cases generate their own random binary files, so these files are so you can test your solutions locally.

Successful output should look like:

```
setilab-handout> Conducting SETI scan for user (<netid1>)...
setilab-handout> Receiving signal from SETI...
```

You should end up with `sig-[0-3].bin` in the `setilab-handout` directory.

2. Band Scan

With your binary files, you can now run `band_scan.c`

```
setilab-handout> make
setilab-handout> ./band_scan bin sig-[].bin 400000 32 10
```

Here, the `text|bin|mmap` argument indicates how the `signal_file` is to be interpreted. `Fs`, `filter_order`, and `num_bands` relate to the problem, and will be given to you. We will describe them a bit later. For the purposes of this lab, you can assume that `Fs` will always be 400000 (400 kHz).

If you run `band_scan` on a signal that has an alien, it will say something like the following:

```
setilab-handout> ./band_scan bin sig.bin 400000 32 10

type:      Binary
file:      sig.bin
Fs:        400000.000000 Hz
order:     32
bands:     10
Load or map file
Read 11999489 samples
signal average power: 0.336757

 0      0.000100 to      19999.999900 Hz:      0.002624 ** (meh)
 1      20000.000100 to      39999.999900 Hz:      0.000108 * (meh)
 2      40000.000100 to      59999.999900 Hz:      0.000060* (meh)
 3      60000.000100 to      79999.999900 Hz:      0.000061 * (meh)
 4      80000.000100 to      99999.999900 Hz:      0.081506 ***** (WOW)
 5     100000.000100 to     119999.999900 Hz:      0.081501 ***** (WOW)
 6     120000.000100 to     139999.999900 Hz:      0.000042 * (meh)
 7     140000.000100 to     159999.999900 Hz:      0.000023 * (meh)
 8     160000.000100 to     179999.999900 Hz:      0.000017 * (meh)
 9     180000.000100 to     199999.999900 Hz:      0.000019 * (meh)

<detailed timing information>
Analysis took 10.719594 seconds
POSSIBLE ALIENS 80000.000100-120000.000100 HZ (CENTER 100000.000000 HZ)
```

This result means that there is an unusual amount of power centered around the radio frequency 100 000 Hz (100 kHz) in this signal.

Try this command out on all your bin files, and see which ones could possibly have aliens!

NOTE: We've checked with our scientists, and they tell us that, for the purpose of this lab, aliens will almost certainly be transmitting on frequencies between 50 kHz and 150 kHz, so if you see some power outside of this band it, it is likely noise or interference and is safe to ignore.

If you've detected a large amount of power in a narrow band, you just might have an alien message! To further verify this, you can see if it has something that might be akin to language by using this program:

```
setilab-handout> ./analyze_signal -p sig.bin
```

The -p option tells it to print out high-powered pulse lengths it found in the signal, and is optional. Do you notice anything special about the pulse lengths you find? If it found something very interesting, it will output the following:

The information content of this signal appears to be very high!

At this point we can pretty much assume we have an alien message on our hands. You can then decode the signal for audio playback:

```
setilab-handout> ./amdemod -c 100000 sig-0.bin
```

Here, the "100000" is the center of the "possible aliens" band you detected using band_scan. amdemod will produce the output file sig-0.out.

You can then convert sig-0.out to a WAV file:

```
setilab-handout> ./bin2wav sig-0.out
```

This will produce the file out.wav, which you can play to hear what your alien sounds like.

STEP 2: P_BAND_SCAN.C

After you implement your own version of `p_band_scan.c` with parallelism and modify the Makefile to produce the `p_band_scan` binary. You can then try running the program to see if the outputs match `band_scan.c`.

Your parallel implementation of `band_scan` will look like this:

```
setilab-handout> ./p_band_scan
usage: p_band_scan text|bin|mmap signal_file Fs filter_order
num_bands num_threads num_processors
```

The two additional arguments denote the number of threads and the number of processors you are to use (processors 0 to `num_processors-1`). You should round-robin your threads over the processors as a starting point.

Your program needs to match the output format of the LAST LINE of `band_scan` exactly.

STEP 3: Submitting via Slurm

To submit this lab, you must submit a job on the compute node with the following command.

```
setilab-handout> ./scripts/seti-run --eval -a <# of file which
contains the alien>
```

You can view your job by running:

```
setilab-handout> ./scripts/seti-look
```

You can cancel your job by running:

```
setilab-handout> ./scripts/seti-cancel
```

Success Criteria and Grading

There is a live leaderboard at <https://leaderboard.seti-lab-cs32.com/>.

To successfully complete this lab, your **name must appear on the leaderboard** and you must **pass the timing checks**.

If your name does not appear on the leaderboard, your submission likely did not pass the correctness checks. Please check the output directory in your lab folder for details about the failure.

Also, your name must not be highlighted in red on the leaderboard — red indicates that the timing criteria were not met.

Additional credit will be awarded to the top performing submission (with full functionality) on the leaderboard. Good luck!

Grading:

- **40 points: Correctness**
 - Code has the same output as `band_scan.c` on 4 random test cases
- **60 points: Performance**
 - Code meets time constraints
- **+25 Extra Credit Points: Performance Leaderboard**
 - Awarded to the **Top 3 students** with the fastest submissions.
 - *Note on Eligibility:* The Top 3 will be finalized on the official assignment due date. While you may use slip days to submit the assignment late, submissions utilizing slip days will **not** be considered for these extra credit points.

APPENDIX

A. BACKGROUND KNOWLEDGE

Signal Processing

To understand this lab, you need to understand some basics of signal processing and how you might parallelize them.

Signal Processing Basics

For the purposes of this lab, a signal is an N element array of double precision floating point numbers, where each number indicates the level of the signal (the intensity of the induced current in the antenna) at a given point in time. A signal like this is more specifically termed a discrete-time signal (from electrical engineering), and is also known as a time series (from statistics). Our discrete-time signals are periodically sampled, meaning that the time between one sample and the next is fixed. The inverse of this sample time is called the sample rate, and it is the F_s in the above. To make this more concrete, suppose we have a sequence of numbers like this in the signal file:

```
0.03
0.50
-0.2
-0.9
-0.6
0.3
```

Let's suppose that the first value (0.03) arrived at time 0, and further suppose the sample rate (F_s) is 1000 per second (or 1000 Hz, or 1 kHz). This means the second value (0.50) arrived at time $0 + (1/1000) = 0.001 = 1$ ms, the next (-0.2) at 2 ms, and so on.

A neat fact of signal processing is that any signal can be represented as a sum of shifted, amplified sine waves at specific frequencies (keyword: Fourier). We can think of the signal as being composed of different "amounts" (amplitudes) of these sine waves, which is essentially what the `band_scan` program computes. Each band consists of a group of sine waves. For reasons not important here (keyword: Nyquist), the highest frequency of such a sine wave we can find is half the frequency of the sample rate, so 500 Hz in the above example.

Now you can better understand what `band_scan` is doing, since you know the F_s that was used (400 kHz), and that resulted in being able to look from 0 Hz to 200 kHz. In the earlier example, we asked `band_scan` to look at the sine waves in that range in 10 bins or bands (0 kHz to 20 kHz, 20 kHz to 40 kHz, and so on). It then found a lot of power (meaning high amplitude sines) around 100 kHz.

`band_scan` does its work by applying band pass filters. A band pass filter passes through sine waves that are in a range of frequencies (the band), and rejects others.

If you look at `band_scan.c`, you'll see that for each band of interest, it first generates a filter design. This form of filter is the simplest variety – it looks like another signal in that it's just an array of M doubles, where $M = \text{filter_order} + 1$. The bigger M (`filter_order`) is, the better the filter is, but the more expensive it is to use. Unless M is truly huge, the time spent in generating the filter is negligible.

Once the filter for the band has been generated, `band_scan.c` next applies it to the signal using a small function called `convolve_and_compute_power()`. `convolve_and_compute_power()` is the function that does the heavy lifting (keyword: convolution). You can easily read the function's code to see what it does, but here is the challenge: if the filter is of size M and the signal of size N , then `convolve_and_compute_power()` does $O(N \cdot M)$ work. As `convolve_and_compute_power()` executes, it also sums up the power in the selected band of the output signal, which is an additional $O(N)$ work. Once all the bands are completed, the band-sums are evaluated using thresholds to see if there is something interesting to be found. This takes negligible work. Therefore, if you are asked to use B bands, the whole program does $O(B \cdot N \cdot M)$ work.

Parallelizing the Signal Processing

Your goal is to do the $O(B \cdot N \cdot M)$ work in less than $O(B \cdot N \cdot M)$ time. Ideally, if you have P processors, you would be able to do it in $O(B \cdot N \cdot M / P)$ time. You may be able to achieve this in some cases. For different values of B , N , M , and P , you may find there are different challenges to achieving high performance. For example, you may be bottlenecked by I/O, just reading/writing the files, or by the CPU, or by the memory system.

Parallelism requires not just that you have the ability to do multiple things at once, but also that no intrinsic ordering or dependencies in the algorithm are violated. If you violate them, the result may be incorrect. Correct and slow is better than fast and wrong, so be careful. So, you are looking at the work to find out what chunks you can correctly do together.

There are at least three independent ways to parallelize `band_scan` that we can think of. You are welcome to try out any approach you think might be interesting. The one approach you definitely will want to try first, however, is simply to execute bands simultaneously, that is, to parallelize across the bands. That should be sufficient for achieving the basic learning and performance goals of the lab.

When we think of the performance of a parallel program, we need to think beyond just the basic run-time for an example. In particular, we are also interested in how the program scales with the problem size and with the number of processors. In a perfectly scalable program, we can always double the number of processors and expect the execution time to be cut in half. Very few parallel programs work this way. In fact, if they do, they are usually called embarrassingly parallel. A good performance measurement of `band_scan` would look at the execution time as a function of:

1. the number of processors, P
2. the signal size, N
3. the size of filter, M

Another useful view is called a speedup curve, where we fix the problem size (N and M), and vary the number of processors, plotting time-with-1-processor/time-with- P -processors as a function of P . Finally, we can simply plot time-with- P -processors as a function of P .

The use of the terms processor, core, hardware thread, software thread (pthread), and process can be a bit confusing.

A machine may have one or more processors. Each processor is a separate chip mounted on the motherboard of the machine. The processors share the main memory system (DRAMs), although each processor can access memory near it faster. A processor can have one or more cores.

A core is a complete execution unit plus one or more levels of memory cache. Each core can independently fetch, decode, and execute instructions. Usually, the cores of a processor share an L2 and/or L3 cache. Each core may have one or more hardware threads. Individual implementations vary across manufacturers and hardware generations. If you are interested in really getting deep into extracting as much performance as possible, there are programming guides available for Intel's Skylake microarchitecture that may prove useful.

A hardware thread (hyperthread is what Intel likes to call this) consists of hardware that can fetch and decode instructions. All the hardware threads of a core share the single execution engine of the core, but each individually have their own registers and control circuitry. The purpose of multiple hardware threads per core is basically to keep that engine busy by feeding it work from multiple sources – if thread #1 is stalled and waiting for data to come back from main memory, the CPU can do work on thread #2, and vice versa. The operating system creates the abstraction of software threads, which are the pthreads you will program in this lab. The OS dynamically maps software threads onto hardware threads. You can ask the OS to map a software thread to a specific hardware thread on a specific core on a specific processor, which prevents competition for resources by spreading the threads out across the CPU.

The OS also creates the abstraction of processes, which contain one or more software threads running in a shared memory space. These processes are accessed by the programmer through fork/wait and similar system calls. The OS implements processes using both software threads and virtual memory management, both of which are tightly coupled with the hardware. In addition, some programming languages (e.g., Scheme, some Java implementations, etc.) implement another level of threads and processes on top of the operating system supplied software threads (pthreads) and processes.

Two other ways you might find it possible to get better raw performance and/or better speedup include:

- Parallelizing the individual convolutions themselves. If you look at the `convolve_and_compute_power()` function, you'll see that it does $O(M)$ work for each of the N output data points. You could work on those data points in parallel instead of, or in addition to working in parallel across the bands.
- Exploring the impact of different compiler optimizations to increase the performance of the `convolve_and_compute_power()` function.

Carrier Frequencies

Each of the signal files has a single signal in it (across a large frequency range). That large range is not what the aliens used in their original signal; they used a much smaller frequency range, we just collected signals in that large range. First we need to remove all the boring relatively low-power noise in the signal, so we filter it first. That filtered signal (which is now the interesting one) still covers a certain bandwidth. To fully answer why the signal covers a certain bandwidth, we need to make a digression into some signal modulation topics.

There are a lot of ways to make a transmissible signal. Remember, signals are sinusoidal and the frequency of the alien signal was chosen by the aliens for some reason. The most obvious way to make a data-carrying signal is to assign 0 the low amplitude and assign 1 the high amplitude (This is how AM (Amplitude Modulation) Radio works.). But the signal's frequency is just too low for real use (your carrier frequencies are in the 10's of kHz range, which would roughly translate to the 10's of kbit/s range). Note that this is kilo BIT per second, so divide by 8 to get kB/s. To make higher data-rate signals, we can just oscillate our data signal faster.

Now a fun and fun part about electromagnetic waves (light, including visible light, radio, x-rays, and so on) comes into play here; they have constructive interference, not destructive. Constructive interference means that light on any two frequencies add together. (Sound has destructive interference, which is why we can have noise-canceling headphones.) So taking our AM radio example from above, we can add together the very slow carrier wave and a very fast data signal to get a modulated signal. Figure 1 shows the effect of optical/EM signal modulation in amplitude.

We have now encoded a very high-speed message onto a very slow carrier signal! So long as the receiver knows what we are doing, they can demodulate the signal to split the actual data signal out from the carrier wave, and get the data back!

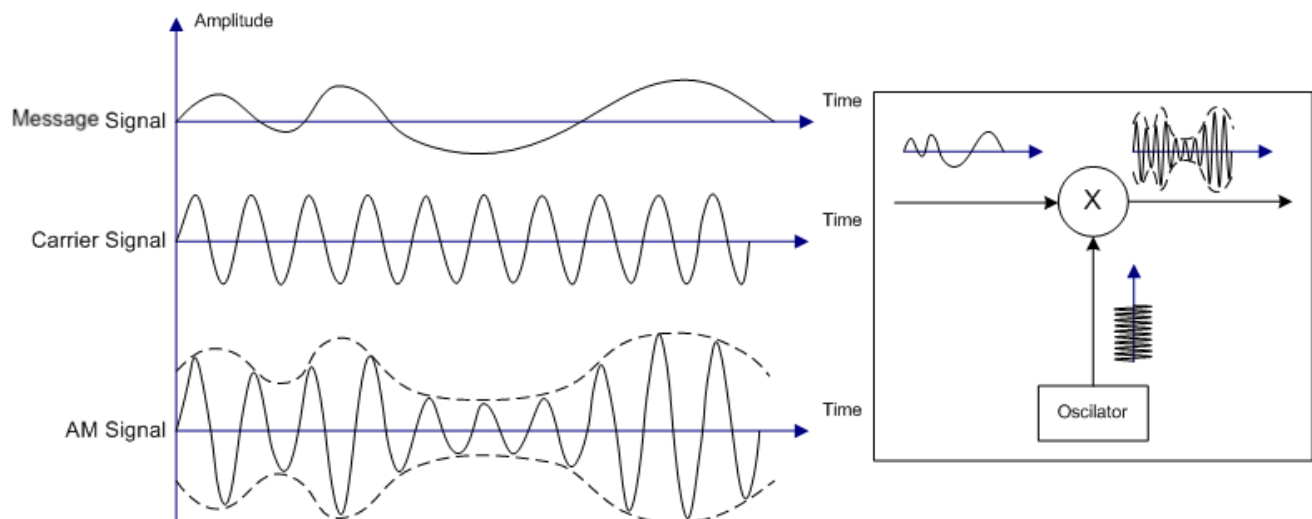
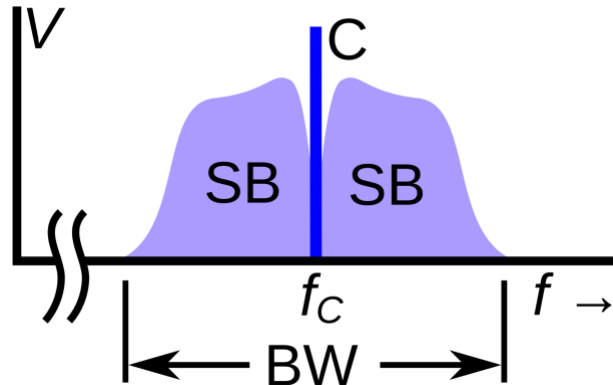


Figure 1: Signal Modulation (Credits to Wikipedia)

So far, we have talked about AM radio, which is the easier-to-understand way to modulate data signals onto carrier frequencies. But AM radio suffers from high signal degradation over medium distances because of its $(1/r^2)$ behavior (See Inverse Square Law) and noisy background things. What if we could modulate our data signal on the frequency of the carrier signal instead? This would avoid the Inverse Square Law problem entirely.

Frequency Modulation (FM) is the name of this technique, and is the basis for FM radio. FM is better because frequency does not degrade nearly as fast over distance compared to amplitude, but we now “spill” into nearby frequencies! Have you ever noticed how FM radio stations are always at least 0.2 MHz away

from each other? This is to account for this “spilling” while still leaving enough empty space between the stations for signals to not garble each other. Figure 2 gives you a visualization of bandwidth (BW), the carrier/center frequency (f_c), and the bottom/top frequencies (lowest & highest edges of SB, respectively).



Regular Unix I/O and Memory-Mapped I/O

You will need to read the binary input signal file. Although code is provided in `signal.c` to do this, you may want to review it to understand what it's doing, and to better choose between the three techniques offered there. The code supports text-based files using the C stdio library (available wherever C is available), regular Unix I/O of binary files, and memory mapped Unix I/O of binary files. Regular I/O consists of the use of the Unix system calls `open`, `read`, `write`, `lseek`, and `close`. The main idea is that regular I/O is explicit—you need to tell the operating system exactly what to do and when to do it using the system calls.

In memory-mapped I/O, we ask the operating system (via the `mmap` system call) to map the file into our address space so that we can treat it like a chunk of data in memory. Recall that in exec-ing a program, the operating system memory maps portions of the executable program file into the address space and then jumps to it. The `mmap` system call gives us access to the same functionality. Memory-mapped I/O is implicit—you just read and write memory, and the operating system translates that into actual I/O as needed.

Pthreads and Processor Affinity

You will need to partition the work among multiple processors. To do this, you will use threads, which are explained in some detail in your book and were also covered in lecture. In Linux, the threading interface is called `pthread`.

The file `pthread-ex.c` shows how to use the basic `pthread` system calls. It is very important that you supply the `-pthread` option to `gcc` when compiling code that uses `pthread` (see the Makefile). The `pthread_create` system call creates a new thread that starts executing in the function you specify. That is, it looks like a function call, but the caller does not wait for the callee to finish! Instead, the caller and callee continue to run simultaneously. The caller can explicitly wait for the callee to finish by using the `pthread_join` system call.

Note that this is quite similar to the `fork` and `wait` system calls discussed in class, the book, and the systems programming handout. However, while `fork` creates an entirely new process that is an independent clone of the parent process, `pthread_create` creates a new thread of execution (that starts at the callee function) **within the current process**. The new thread of execution shares all the memory and other state of the current process with the thread that created it, and with all of the other threads in the process. This means they must carefully coordinate access to the shared memory to avoid serious and difficult to track down bugs. While this is extremely challenging, it is outside the scope of this lab. In this lab, your threads

only need to *read* from shared memory (the input signal). Their writes to the output signal do not need to overlap.

You can create as many threads as you want (and that the operating system has memory to track). The operating system will interleave the execution of these threads in time and across the processors available on the system. That is, the operating system can switch from thread to thread on any given processor, and it can move a thread from one processor to another. This scheduling activity happens on the order of every millisecond or so. It does this to “balance the load” and to maintain fairness among all the threads in the system. However, it is sometimes convenient, especially in a parallel program, to directly control which processor a thread runs on. This is known as processor affinity. A thread can advise the operating system of the set of processors it would like to be run on. The `pthread-ex.c` example shows how a thread can request that it only run on a specific processor.

The `parallel-sum-ex.c` example shows how to use pthreads to sum up an array of doubles in parallel.

Measuring Time, Performance, and Resource Usage

The lab code `timing.[ch]` includes three timing tools. The `band_scan.c` code uses them to measure its own activity. You can use them to find out what the bottlenecks are in your code.

The first timing tool, `get_seconds()`, is measuring the passage of real time using the Unix `gettimeofday()` system call. This can be used arbitrarily in any thread since there is exactly one time across the whole system.

The second timing tool, `get_cycle_count()`, is measuring the passage of real time using the processor cycle counter. This is the most accurate measurement, but it’s important to note that **each core has its own cycle counter**, which can make for confusion if a thread migrates from one core to another.

The third timing tool, `get_resources()`, measures resource usage, including time spent using the processor. You can measure the resource usage of your entire process, and of individual threads. This mechanism gives highly detailed information, but the resolution is much lower than the other timing tools.

Advice

We document a suggested approach to this lab below. There are other ways to work with and solve this problem.

The biggest thing to keep in mind is: **don’t panic!**

There is a lot here, which can be overwhelming if you’ve never been giving an existing codebase to work on. Part of what we’re trying to do is give you the experience of working within an existing, significant codebase. Remember that what you’re doing here is writing `p_band_scan.c`. There is a lot of complexity in this overall system, but you are remember that you are not writing it from scratch — you’re just parallelizing this one component of it. Our own reference implementation of `p_band_scan.c` is based on `band_scan.c` and consists of only about 180 additional lines of C. Ask questions and get help.

1. Read and play with the handout code to get a good sense of how things work. Make sure you try both the `alien_msg.sig` and `noalien.sig` signals, and to get your own signals using `siggen`.
2. Read through `band_scan.c` and `filter.c`, focusing specifically on the `analyze_signal()` and `convolve_and_compute_power()` functions, which do the main work.

3. Understand the `pthread-ex.c` and `parallel-sum-ex.c` code. Start by just compiling it and running it, and then look at it more carefully. `parallel-sum-ex.c` shows how to sum up the elements of an array in parallel.
4. Develop a strategy to parallelize the `band_scan` processing. The simplest approach will work fine.
5. Implement your design of `p_band_scan` using `pthreads`. It must be possible to specify how many threads to use and how many processors to use at run-time. Note that you will need to change the Makefile, as documented in Section 4.3. You will find comments in the Makefile that describe how to do this. Your Makefile rule to build `p_band_scan` must be called `p_band_scan`.
6. Test your program to make sure it is correct (compare against the sequential `band_scan` program). For basic testing, you can directly run `p_band_scan` and compare against the results from `band_scan`.